3. 内核补上无法只靠库函数实现的语义，如 `lseek()`，以及 `socket()` → `bind()` → `listen()` → `accept()` 的生命周期。

表 8.3: POSIX 接口到 xv6 的映射

POSIX 接口	实现
`open/read/write/lseek/fstat`	直接对应系统调用；`O_CREAT` 是 `O_CREATE` 的别名
`pthread_create/join/exit`	`clone/tjoin/texit`；`pthread_join` 同时回收用户栈
`pthread_mutex_*`、`sem_*`	内核同步对象；支持 `PTHREAD_MUTEX_INITIALIZER` 延迟初始化
`clock_gettime(CLOCK_MONOTONIC)`	`clock_us()`，读 `CNTVCT_EL0`
`sleep/usleep/nanosleep`	秒语义；原来按 tick 计的 `sleep` 改名为 `sleep_ticks`
`socket/bind/listen/accept`	统一的 socket 文件描述符，可用 `read/write/close/epoll`

`posixtest` 只包含标准头文件，验证文件读写与 `lseek`、POSIX 信号量、两个 pthread 并发更新共享变量、静态初始化的互斥锁以及 socket 生命周期。

这仍是第一阶段：系统调用失败只返回 −1，没有传回具体错误码，`errno` 也还不是线程局部的；缺少 `waitpid`、`dup2`、条件变量、目录流和 TCP `connect()`。

8.6 在 xv6 里编译 C 程序

`tcc` 是一个面向教学的极小 C 编译器，运行在 xv6 用户态，直接输出 `exec()` 能加载的 ELF64 文件，不需要汇编器和链接器：

```
$ edit hello.c
int main() {
  int answer = 6 * 7;
  printf("hello from xv6 C\n");
  return answer - 42;
}
$ tcc hello.c -o hello
$ hello
hello from xv6 C
```

它支持整数局部变量、四则运算、`return`、只含字面量的 `printf/puts` 和注释。生成的代码用 `x29` 指向 256 字节的局部变量区，表达式结果放在 `x0`，字符串地址用 `adr` 指令取得，所以无需重定位；输出和退出直接用 `svc #0` 调用 `SYS_write` 和 `SYS_exit`。文件布局是一个 ELF 头、一个程序头、机器码和字符串池。

一个能在自己身上编译并运行程序的系统，才算是有了一点“自举”的味道。

8.7 一个大型用户程序：sshd

`sshd` 有一千多行，是整个系统里最复杂的用户程序，也是对内核各子系统的一次综合考验。它用到了：

- TCP socket 与 `accept`——网络协议栈；
- 密码学全部在用户态完成：基于 TweetNaCl[3] 的 X25519 密钥交换和 Ed25519 主机密钥签名，传输层用 AES-128-CTR 加密、HMAC-SHA256 校验；熵来自内核的硬件随机数发生器（`kernel/rng.c`）；
- `/etc/passwd` 口令认证；
- PTY：为每个会话分配一对伪终端，shell 的 fd 0/1/2 指向 PTY 从端，`sshd` 读写主端并加密转发；
- 会话与控制终端：SSH 会话中的 `/dev/tty` 指向该会话的 PTY。

调试 `sshd` 时发现的问题推动了内核的多处改进：字符回显慢暴露了“网卡每 100 ms 轮询一次”和“TCP `write()` 同步等 ACK”两个瓶颈，促成了 SDIO 中断、workqueue 下半部、NAPI 和 TCP 异步发送队列；`ps` 输出